

Reusable Component Library

All reusable components live in `src/components/`. Each component is fully typed with TypeScript and styled with NativeWind.

Table of Contents

1. [GuardCard](#)
 2. [BookingCard](#)
 3. [MapGuardMarker](#)
 4. [OTPInput](#)
 5. [RatingStars](#)
 6. [StatusBadge](#)
 7. [SOSButton](#)
 8. [LoadingOverlay](#)
-

1. GuardCard

src/components/GuardCard.tsx

```
import { View, Text, Image, TouchableOpacity } from 'react-native';
import { Star, MapPin, Shield } from 'lucide-react-native';
import type { Guard } from '@types';
import { GuardTier } from '@types';

interface GuardCardProps {
  guard: Guard;
  onBook: (guard: Guard) => void;
}

const TIER_COLORS: Record<GuardTier, string> = {
  [GuardTier.BASIC]: '#64748b',
  [GuardTier.STANDARD]: '#3b82f6',
  [GuardTier.PREMIUM]: '#8b5cf6',
  [GuardTier.ELITE]: '#f59e0b',
};

const formatDistance = (metres: number): string => {
  if (metres < 1000) return `${Math.round(metres)}m`;
  return `${(metres / 1000).toFixed(1)}km`;
```

```

};

export function GuardCard({ guard, onBook }: GuardCardProps) {
  const tierColor = TIER_COLORS[guard.tier];

  return (
    <View className="bg-slate-800 rounded-2xl p-4 flex-row items-
center mb-3">
      {/* Photo */}
      <Image
        source={{
          uri: guard.profilePhoto ?? 'https://via.placeholder.com/
56',
        }}
        className="w-14 h-14 rounded-xl mr-3"
      />

      {/* Info */}
      <View className="flex-1">
        <View className="flex-row items-center gap-2 mb-0.5">
          <Text className="text-white font-semibold text-base"
numberOfLines={1}>
            {guard.name}
          </Text>
          {/* Tier badge */}
          <View
            className="px-2 py-0.5 rounded-full"
            style={{ backgroundColor: tierColor + '33' }}
          >
            <Text
              className="text-[10px] font-semibold capitalize"
              style={{ color: tierColor }}
            >
              {guard.tier}
            </Text>
          </View>
        </View>

        {/* Rating */}
        <View className="flex-row items-center gap-1 mb-1">
          <Star size={12} color="#f59e0b" fill="#f59e0b" />
          <Text className="text-slate-300 text-xs">
            {guard.rating.toFixed(1)}{' '}
          <Text className="text-
slate-500">({guard.totalReviews})</Text>
          </Text>
        </View>

        {/* Distance */}
        {guard.distance !== undefined && (
          <View className="flex-row items-center gap-1">
            <MapPin size={11} color="#64748b" />
            <Text className="text-slate-500 text-xs">
              {formatDistance(guard.distance)} away
            </Text>
          </View>
        )}
      </View>
    </View>
  );
}

```

```

        </Text>
      </View>
    )}
  </View>

  { /* Availability dot + Book button */ }
  <View className="items-end gap-2">
    <View className="flex-row items-center gap-1">
      <View
        className={`w-2 h-2 rounded-full ${
          guard.isAvailable ? 'bg-green-400' : 'bg-slate-600'
        }`}
      />
    </View>
    <TouchableOpacity
      onPress={() => onBook(guard)}
      disabled={!guard.isAvailable}
      className={`px-3 py-2 rounded-lg ${
        guard.isAvailable ? 'bg-blue-600' : 'bg-slate-700'
      }`}
      activeOpacity={0.8}
    >
      <Text
        className={`text-xs font-semibold ${
          guard.isAvailable ? 'text-white' : 'text-slate-500'
        }`}
      >
        {guard.isAvailable ? 'Book' : 'Busy'}
      </Text>
    </TouchableOpacity>
  </View>
</View>
);
}

```

2. BookingCard

src/components/BookingCard.tsx

```

import { View, Text, TouchableOpacity } from 'react-native';
import { ChevronRight, Shield } from 'lucide-react-native';
import { StatusBadge } from './StatusBadge';
import type { Booking } from '@types';
import { formatCurrency, formatDateTime } from '@utils/format';

interface BookingCardProps {
  booking: Booking;
  onPress: (booking: Booking) => void;
}

export function BookingCard({ booking, onPress }:

```

```

BookingCardProps) {
  return (
    <TouchableOpacity
      onPress={() => onPress(booking)}
      className="bg-slate-800 rounded-2xl p-4"
      activeOpacity={0.75}
    >
      <View className="flex-row items-start justify-between mb-3">
        {/* Booking ID + status */}
        <View>
          <Text className="text-slate-400 text-xs mb-1">
            Booking #{booking.id}
          </Text>
          <StatusBadge status={booking.status} />
        </View>
        <ChevronRight size={18} color="#475569" />
      </View>

      {/* Guard */}
      <View className="flex-row items-center gap-2 mb-2">
        <View className="w-8 h-8 bg-slate-700 rounded-full items-center justify-center">
          <Shield size={14} color="#3b82f6" />
        </View>
        <Text className="text-white text-sm font-medium">
          {booking.guard?.name ?? 'Guard not assigned'}
        </Text>
      </View>

      {/* Date + amount */}
      <View className="flex-row items-center justify-between">
        <Text className="text-slate-500 text-xs">
          {formatDateTime(booking.createdAt)}
        </Text>
        {booking.totalAmount > 0 && (
          <Text className="text-white text-sm font-semibold">
            {formatCurrency(booking.totalAmount)}
          </Text>
        )}
      </View>
    </TouchableOpacity>
  );
}

```

3. MapGuardMarker

src/components/MapGuardMarker.tsx

```

import { useEffect, useRef } from 'react';
import { View, Text, Animated } from 'react-native';
import { Marker } from 'react-native-maps';

```

```

import { Shield } from 'lucide-react-native';
import type { Guard } from '@types';
import { GuardTier } from '@types';

interface MapGuardMarkerProps {
  guard: Guard;
  onPress: (guard: Guard) => void;
}

const TIER_COLORS: Record<GuardTier, string> = {
  [GuardTier.BASIC]: '#64748b',
  [GuardTier.STANDARD]: '#3b82f6',
  [GuardTier.PREMIUM]: '#8b5cf6',
  [GuardTier.ELITE]: '#f59e0b',
};

export function MapGuardMarker({ guard, onPress }:
MapGuardMarkerProps) {
  const pulseAnim = useRef(new Animated.Value(1)).current;
  const color = TIER_COLORS[guard.tier];

  useEffect(() => {
    if (guard.isAvailable) {
      const animation = Animated.loop(
        Animated.sequence([
          Animated.timing(pulseAnim, {
            toValue: 1.3,
            duration: 800,
            useNativeDriver: true,
          }),
          Animated.timing(pulseAnim, {
            toValue: 1,
            duration: 800,
            useNativeDriver: true,
          }),
        ]),
      );
      animation.start();
      return () => animation.stop();
    }
  }, [guard.isAvailable]);

  if (!guard.location) return null;

  return (
    <Marker
      coordinate={{
        latitude: guard.location.latitude,
        longitude: guard.location.longitude,
      }}
      onPress={() => onPress(guard)}
      tracksViewChanges={false}
    >
    <View className="items-center">

```

```

    { /* Pulse ring */
    {guard.isAvailable && (
      <Animated.View
        style={{
          position: 'absolute',
          width: 36,
          height: 36,
          borderRadius: 18,
          backgroundColor: color + '40',
          transform: [{ scale: pulseAnim }],
        }}
      />
    )}

    { /* Icon container */
    <View
      style={{ backgroundColor: color }}
      className="w-9 h-9 rounded-full items-center justify-
center shadow-md"
    >
      <Shield size={16} color="#fff" />
    </View>

    { /* Name label */
    <View className="bg-slate-900/90 rounded-md px-1.5 py-0.5
mt-1">
      <Text className="text-white text-[9px] font-medium">
        {guard.name.split(' ')[0]}
      </Text>
    </View>
    </View>
    </Marker>
  );
}

```

4. OTPInput

src/components/OTPInput.tsx

```

import { useRef, useCallback } from 'react';
import {
  View,
  TextInput,
  NativeSyntheticEvent,
  TextInputKeyPressEventData,
  Clipboard,
} from 'react-native';

interface OTPInputProps {
  length: number;
  value: string;

```

```

    onChange: (value: string) => void;
    disabled?: boolean;
  }

export function OTPInput({ length, value, onChange, disabled =
false }: OTPInputProps) {
  const inputs = useRef<Array<TextInput | null>>([]);

  const digits = Array.from({ length }, (_, i) => value[i] ?? '');

  const handleChange = useCallback(
    (text: string, index: number) => {
      // Handle paste: if text length > 1, fill all from this
index
      if (text.length > 1) {
        const pasted = text.replace(/\D/g, '').slice(0, length);
        onChange(pasted.padEnd(length, '').slice(0, length));
        const nextIndex = Math.min(pasted.length, length - 1);
        inputs.current[nextIndex]?.focus();
        return;
      }

      const cleaned = text.replace(/\D/g, '');
      const newValue = digits.map((d, i) => (i === index ?
cleaned : d)).join('');
      onChange(newValue.slice(0, length));

      // Move to next input
      if (cleaned && index < length - 1) {
        inputs.current[index + 1]?.focus();
      }
    },
    [digits, length, onChange],
  );

  const handleKeyPress = useCallback(
    (
      e: NativeSyntheticEvent<TextInputKeyPressEventData>,
      index: number,
    ) => {
      if (e.nativeEvent.key === 'Backspace' && !digits[index] &&
index > 0) {
        // Move focus back and clear previous
        const newValue = digits.map((d, i) => (i === index - 1 ?
'' : d)).join('');
        onChange(newValue);
        inputs.current[index - 1]?.focus();
      }
    },
    [digits, onChange],
  );

  return (
    <View className="flex-row gap-3 justify-center">

```

```

    {Array.from({ length }).map((_, index) => (
      <TextInput
        key={index}
        ref={(ref) => {
          inputs.current[index] = ref;
        }}
        value={digits[index]}
        onChangeText={(text) => handleChange(text, index)}
        onKeyPress={(e) => handleKeyPress(e, index)}
        keyboardType="number-pad"
        maxLength={index === 0 ? length : 1} // allow paste on
first box
        editable={!disabled}
        selectTextOnFocus
        caretHidden
        className={`w-12 h-14 text-center text-white text-xl
font-bold rounded-xl border-2 ${
          digits[index]
            ? 'bg-blue-600/20 border-blue-500'
            : 'bg-slate-800 border-slate-700'
          } ${disabled ? 'opacity-50' : ''}`}
      />
    ))}
  </View>
);
}

```

5. RatingStars

src/components/RatingStars.tsx

```

import { View, TouchableOpacity } from 'react-native';
import { Star } from 'lucide-react-native';

interface RatingStarsProps {
  rating: number;
  interactive: boolean;
  size?: number;
  onRate?: (rating: number) => void;
}

export function RatingStars({
  rating,
  interactive,
  size = 20,
  onRate,
}: RatingStarsProps) {
  const stars = Array.from({ length: 5 }, (_, i) => i + 1);

  return (
    <View className="flex-row gap-0.5">

```



```

    {stars.map((star) => {
      const filled = star <= Math.round(rating);
      const icon = (
        <Star
          key={star}
          size={size}
          color="#f59e0b"
          fill={filled ? '#f59e0b' : 'transparent'}
        />
      );

      if (interactive && onRate) {
        return (
          <TouchableOpacity
            key={star}
            onPress={() => onRate(star)}
            hitSlop={{ top: 4, bottom: 4, left: 4, right: 4 }}
            activeOpacity={0.7}
          >
            {icon}
          </TouchableOpacity>
        );
      }

      return <View key={star}>{icon}</View>;
    })}
  </View>
);
}

```

6. StatusBadge

src/components/StatusBadge.tsx

```

import { View, Text } from 'react-native';
import { BookingStatus } from '@types';

interface StatusBadgeProps {
  status: BookingStatus;
}

const STATUS_CONFIG: Record<
  BookingStatus,
  { label: string; bg: string; text: string }
> = {
  [BookingStatus.PENDING]: {
    label: 'Pending',
    bg: 'bg-yellow-500/20',
    text: 'text-yellow-400',
  },
  [BookingStatus.GUARD_ASSIGNED]: {

```

```

    label: 'Assigned',
    bg: 'bg-blue-500/20',
    text: 'text-blue-400',
  },
  [BookingStatus.GUARD_EN_ROUTE]: {
    label: 'En Route',
    bg: 'bg-blue-500/20',
    text: 'text-blue-400',
  },
  [BookingStatus.GUARD_ARRIVED]: {
    label: 'Arrived',
    bg: 'bg-indigo-500/20',
    text: 'text-indigo-400',
  },
  [BookingStatus.ACTIVE]: {
    label: 'Active',
    bg: 'bg-green-500/20',
    text: 'text-green-400',
  },
  [BookingStatus.COMPLETED]: {
    label: 'Completed',
    bg: 'bg-slate-600/40',
    text: 'text-slate-300',
  },
  [BookingStatus.CANCELLED]: {
    label: 'Cancelled',
    bg: 'bg-red-500/20',
    text: 'text-red-400',
  },
},
};

export function StatusBadge({ status }: StatusBadgeProps) {
  const config = STATUS_CONFIG[status];
  return (
    <View className={`px-2.5 py-1 rounded-full self-start $
{config.bg}`}>
      <Text className={`text-xs font-semibold ${config.text}`}>
        {config.label}
      </Text>
    </View>
  );
}

```

7. SOSButton

src/components/SOSButton.tsx

```

import { useRef, useCallback } from 'react';
import { View, Text, Animated, TouchableOpacity } from 'react-
native';
import * as Haptics from 'expo-haptics';

```

```

interface SOSButtonProps {
  onTriggered: () => void;
  holdDuration?: number;
  size?: number;
}

export function SOSButton({
  onTriggered,
  holdDuration = 3000,
  size = 120,
}: SOSButtonProps) {
  const progressAnim = useRef(new Animated.Value(0)).current;
  const scaleAnim = useRef(new Animated.Value(1)).current;
  const holdTimer = useRef<ReturnType<typeof setTimeout> |
null>(null);
  const holdAnim = useRef<Animated.CompositeAnimation |
null>(null);

  const borderRadius = size / 2;

  const handlePressIn = useCallback(() => {
    Haptics.impactAsync(Haptics.ImpactFeedbackStyle.Heavy);

    // Scale up
    Animated.spring(scaleAnim, {
      toValue: 1.06,
      useNativeDriver: true,
    }).start();

    // Radial progress
    holdAnim.current = Animated.timing(progressAnim, {
      toValue: 1,
      duration: holdDuration,
      useNativeDriver: false,
    });
    holdAnim.current.start();

    holdTimer.current = setTimeout(() => {
      Haptics.notificationAsync(Haptics.NotificationFeedbackType.Warning,
        onTriggered());
    }, holdDuration);
  }, [holdDuration, onTriggered]);

  const handlePressOut = useCallback(() => {
    holdAnim.current?.stop();
    holdTimer.current && clearTimeout(holdTimer.current);
    progressAnim.setValue(0);

    Animated.spring(scaleAnim, {
      toValue: 1,
      useNativeDriver: true,
    }).start();
  }, [holdAnim, holdTimer, progressAnim]);

```

```

    Haptics.impactAsync(Haptics.ImpactFeedbackStyle.Light);
  }, []);

  const ringOpacity = progressAnim.interpolate({
    inputRange: [0, 0.1, 1],
    outputRange: [0.3, 0.9, 1],
  });
  const ringScale = progressAnim.interpolate({
    inputRange: [0, 1],
    outputRange: [1, 1.25],
  });

  return (
    <View className="items-center justify-center">
      {/* Animated outer ring */}
      <Animated.View
        style={{
          position: 'absolute',
          width: size + 24,
          height: size + 24,
          borderRadius: borderRadius + 12,
          borderWidth: 3,
          borderColor: '#ef4444',
          opacity: ringOpacity,
          transform: [{ scale: ringScale }],
        }}
      />

      <Animated.View style={{ transform: [{ scale: scaleAnim }] }}
    >
      <TouchableOpacity
        onPressIn={handlePressIn}
        onPressOut={handlePressOut}
        activeOpacity={1}
        style={{
          width: size,
          height: size,
          borderRadius,
          backgroundColor: '#dc2626',
          alignItems: 'center',
          justifyContent: 'center',
          shadowColor: '#ef4444',
          shadowOpacity: 0.5,
          shadowRadius: 16,
          elevation: 10,
        }}
      >
        <Text
          style={{
            color: '#fff',
            fontSize: size * 0.3,
            fontWeight: '900',
          }}
        />
      </TouchableOpacity>
    >
  );

```

```

        >
        SOS
      </Text>
    </TouchableOpacity>
  </Animated.View>
</View>
);
}

```

8. LoadingOverlay

src/components/LoadingOverlay.tsx

```

import { View, ActivityIndicator, Text } from 'react-native';

interface LoadingOverlayProps {
  message?: string;
  transparent?: boolean;
}

export function LoadingOverlay({
  message,
  transparent = false,
}: LoadingOverlayProps) {
  return (
    <View
      className={`flex-1 items-center justify-center ${
        transparent ? 'bg-black/60' : 'bg-slate-900'
      }`}
      style={
        transparent
          ? { position: 'absolute', top: 0, left: 0, right: 0,
bottom: 0, zIndex: 999 }
          : undefined
      }
    >
      <View className="bg-slate-800 rounded-2xl px-8 py-6 items-
center shadow-xl">
        <ActivityIndicator size="large" color="#3b82f6" />
        {message && (
          <Text className="text-slate-300 text-sm mt-3 text-
center">
            {message}
          </Text>
        )}
      </View>
    </View>
  );
}

```